```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"

try:
    df = pd.read_csv(
        url,
        parse_dates=["Month"],
        index_col="Month"
    )
except Exception:
    from google.colab import files
    uploaded = files.upload()

    df = pd.read_csv(
        next(iter(uploaded)),
        parse_dates=["Month"],
        index_col="Month"
    )

df.columns = ["Passengers"]

print("First five records:")
display(df.head())

print("Last five records:")
display(df.tail())

print("Shape:", df.shape)

display(df.describe())
```

```
First five records:
```

Month	Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

```
Last five records:
```

Month	Passengers
1960-08-01	606
1960-09-01	508
1960-10-01	461
1960-11-01	390
1960-12-01	432

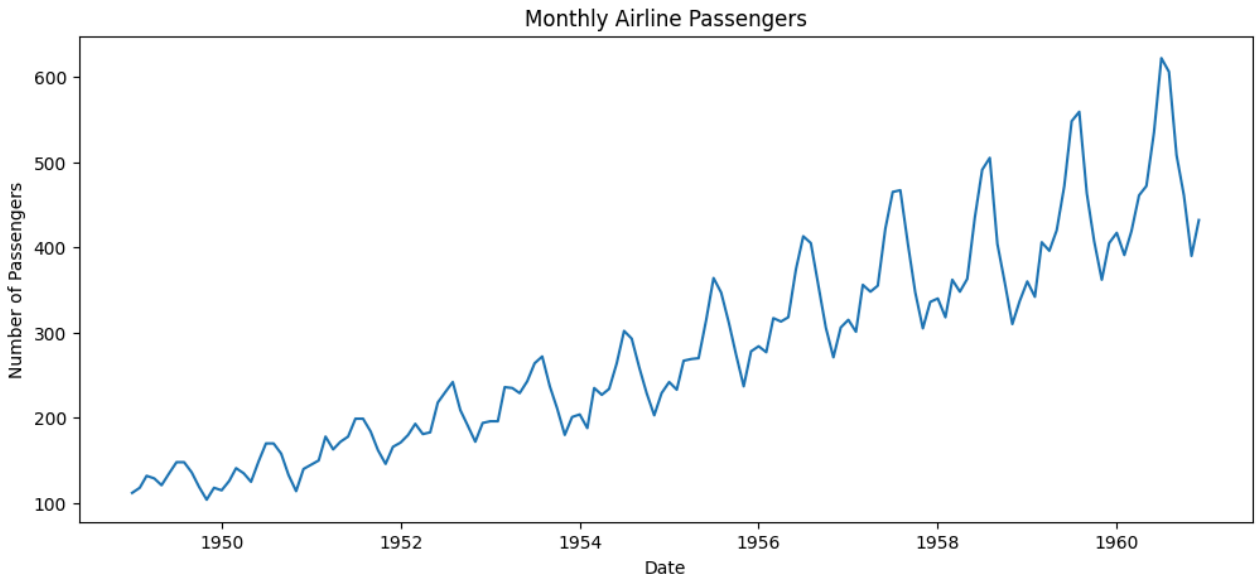
```
Shape: (144, 1)
```

	Passengers
count	144.000000
mean	280.298611
std	119.966317
min	104.000000
25%	180.000000
50%	265.500000
75%	360.500000
max	622.000000

```
plt.figure(figsize=(12, 5))

plt.plot(df.index, df["Passengers"])
```

```
plt.title("Monthly Airline Passengers")
plt.xlabel("Date")
plt.ylabel("Number of Passengers")
plt.show()
```




```
from statsmodels.tsa.stattools import adfuller

result = adfuller(df["Passengers"])

print("ADF Statistic:", result[0])
print("p-value:", result[1])

if result[1] < 0.05:
    print("The series is stationary.")
else:
    print("The series is non-stationary.")
```

```
ADF Statistic: 0.8153688792060498
p-value: 0.991880243437641
The series is non-stationary.
/tmp/ipykernel_1951/4172819932.py:3: FutureWarning: adfuller currently returns a plain tuple whose length depends on the stc
  result = adfuller(df["Passengers"])
```

```
df["First_Difference"] = df["Passengers"].diff()

plt.figure(figsize=(12, 5))

plt.plot(
    df.index,
    df["First_Difference"]
)

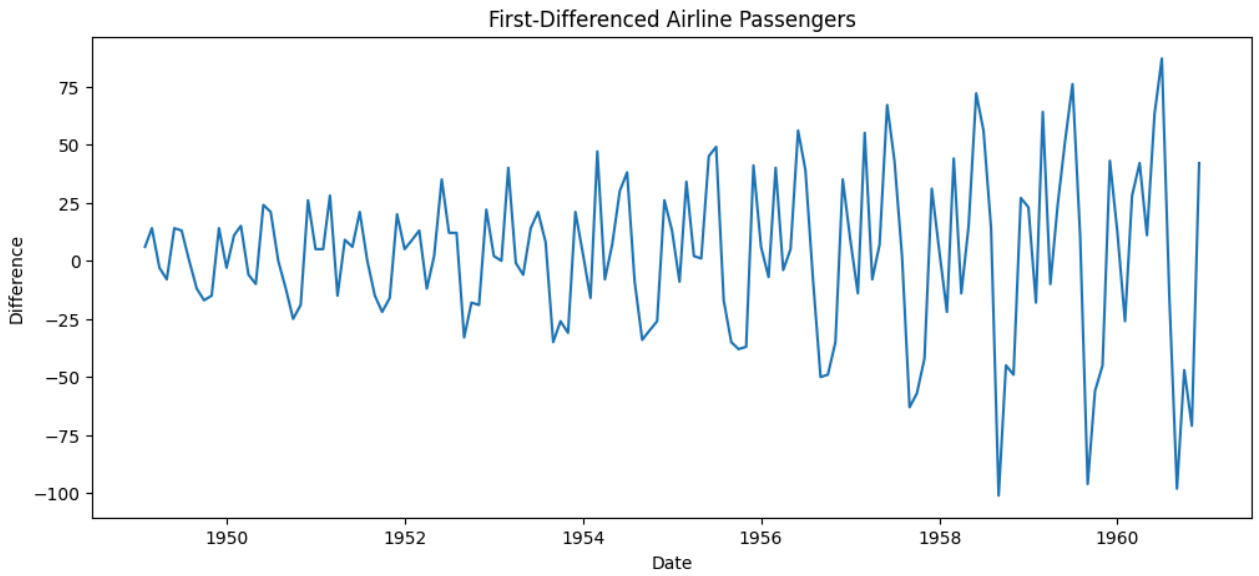
plt.title("First-Differenced Airline Passengers")
plt.xlabel("Date")
plt.ylabel("Difference")
plt.show()

diff_series = df["First_Difference"].dropna()

result = adfuller(diff_series)

print("ADF Statistic after differencing:", result[0])
print("p-value after differencing:", result[1])

if result[1] < 0.05:
    print("Differenced series is stationary.")
else:
    print("Differenced series is still non-stationary.")
```




```
ADF Statistic after differencing: -2.8292668241700047
p-value after differencing: 0.05421329028382478
Differenced series is still non-stationary.
/tmp/ipykernel_1951/2295442402.py:17: FutureWarning: adfuller currently returns a plain tuple whose length depends on the st
  result = adfuller(diff_series)
```

```
from sklearn.metrics import (
    mean_squared_error,
    mean_absolute_error
)

series = df["Passengers"]

split = int(len(series) * 0.8)

train = series.iloc[:split]
test = series.iloc[split:]

print("Training size:", len(train))
print("Testing size:", len(test))
```

```
Training size: 115
Testing size: 29
```

```
from statsmodels.tsa.arima.model import ARIMA

# d=1 because the first difference was applied
model = ARIMA(train, order=(2, 1, 2))

model_fit = model.fit()

forecast = model_fit.forecast(steps=len(test))

print("Forecast values:")
display(forecast)
```

```
Forecast values:
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was
  self._init_dates(dates, freq)
```

	predicted_mean
1958-08-01	487.825560
1958-09-01	460.796800
1958-10-01	451.130922
1958-11-01	441.145635
1958-12-01	435.433346
1959-01-01	431.042077
1959-02-01	428.161471
1959-03-01	426.109439
1959-04-01	424.710071
1959-05-01	423.733689
1959-06-01	423.060606
1959-07-01	422.593654
1959-08-01	422.270786
1959-09-01	422.047151
1959-10-01	421.892392
1959-11-01	421.785245
1959-12-01	421.711080
1960-01-01	421.659739
1960-02-01	421.624199
1960-03-01	421.599597
1960-04-01	421.582567
1960-05-01	421.570778
1960-06-01	421.562617
1960-07-01	421.556968
1960-08-01	421.553058
1960-09-01	421.550351
1960-10-01	421.548477
1960-11-01	421.547180
1960-12-01	421.546282

dtype: float64

```
mse = mean_squared_error(test, forecast)
mae = mean_absolute_error(test, forecast)
rmse = np.sqrt(mse)

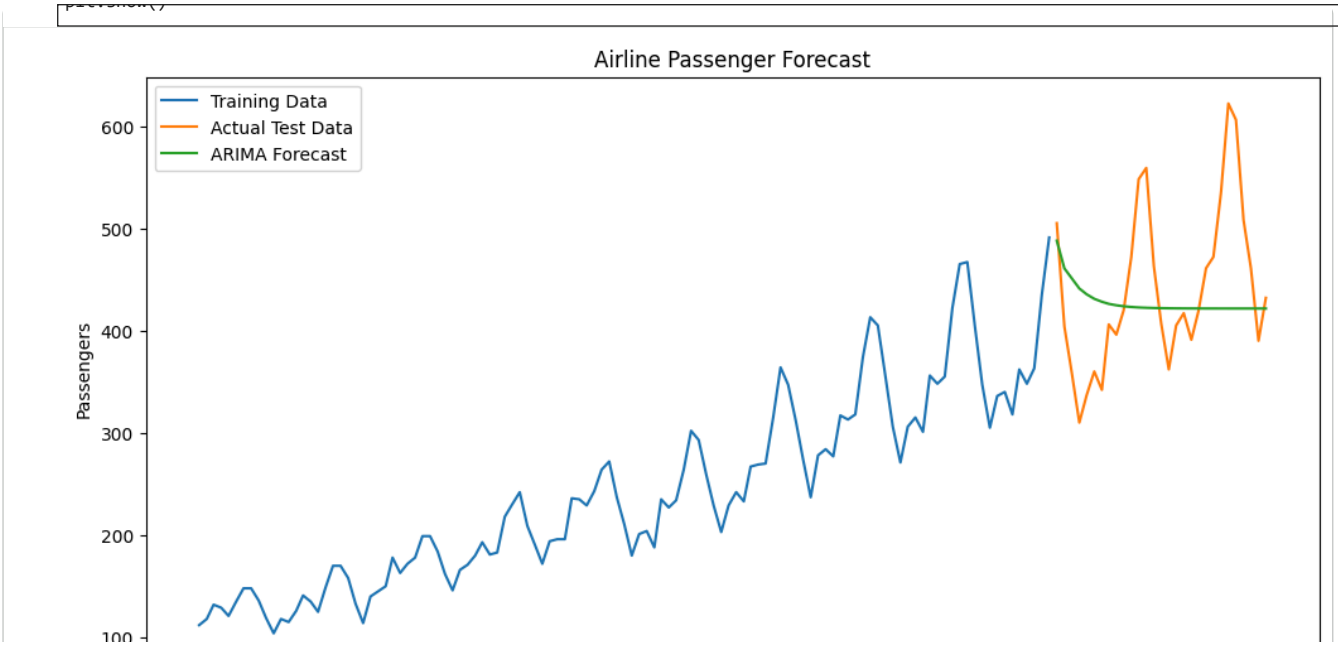
print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)
```

```
Mean Squared Error (MSE): 6808.397034418323
Mean Absolute Error (MAE): 63.54531127532635
Root Mean Squared Error (RMSE): 82.51301130378361
```

```
plt.figure(figsize=(12, 6))

plt.plot(train.index, train, label="Training Data")
plt.plot(test.index, test, label="Actual Test Data")
plt.plot(test.index, forecast, label="ARIMA Forecast")

plt.title("Airline Passenger Forecast")
plt.xlabel("Date")
plt.ylabel("Passengers")
plt.legend()
plt.show()
```


Airline Passenger Forecast
Training Data
Actual Test Data
ARIMA Forecast
Passengers
600
500
400
300
200
100